ONE-DAY DEVELOPMENT PLAN — UPDATED

Support Message Automation System

Locked decisions

- WhatsApp integration: OpenWA (`@open-wa/wa-automate`)
- Dashboard: Next.js + TypeScript + Tailwind + shadcn/ui
- Worker: dedicated Node.js/TypeScript process
- Database: PostgreSQL + Prisma
- Deployment: Docker Compose
- Platforms: Windows + Docker Desktop, Ubuntu + Docker Engine
- Default classification: local rule engine, no AI API per message
- Notifications: Microsoft Teams and optional configured WhatsApp Support Group
- Safety default: SAFE_AUTO_REPLY

Model strategy

- Sonnet 5: 75–80% main implementation
- Fable 5: 15–20% small tasks, documentation, UI polish
- Opus 4.8: 5–10% architecture and genuinely hard problems only

Before starting

1. Create the project folder.
2. Place the updated Master Prompt in the repository.
3. Initialize Git.
4. Work phase by phase.
5. At the end of each phase: build, test, inspect logs, save stable state, commit.

Phase 0 — Architecture Review

Model: Opus 4.8

Prompt:

Review the complete updated project specification. Do not write the application yet. Lock a practical architecture for:

- Next.js dashboard
- dedicated OpenWA worker
- OpenWA provider abstraction and browser/runtime dependencies
- PostgreSQL + Prisma
- Docker Compose

- persistent WhatsApp session
- internal team member filtering
- unified automation rules
- previous/last sender rules
- auto-reply safety
- database-backed queue and idempotency
- Teams and optional WhatsApp internal notifications
- duplicate prevention and recovery

Return:
- final architecture
- repository structure
- Docker services
- database design
- development roadmap
- high-risk areas
- exact implementation order

Stop after the architecture is approved.

Phase 1 — Project Foundation

Model: Sonnet 5

Implement only:
- organized repository/monorepo
- Next.js application
- dedicated worker
- TypeScript
- Docker Compose
- PostgreSQL
- Prisma
- .env.example
- health checks
- persistent volumes
- restart policies

Required services:
- app
- worker
- postgres

Do not implement OpenWA, automation rules, or notifications yet.

Test:

- docker compose up -d --build
- docker compose ps
- app healthy
- worker healthy
- postgres healthy

Commit only when stable.

Phase 2 — Database and Core Models

Model: Sonnet 5

Implement:

- Users
- WhatsAppAccounts
- WhatsAppGroups
- Messages
- InternalTeamMembers
- AutomationRules
- MessageActions / AutomationExecutions
- Notifications
- ProcessingCheckpoints
- AutomationSettings
- SystemLogs

Critical constraints:

- unique WhatsApp account + WhatsApp message ID
- unique action idempotency key
- indexes for sender, group, status, timestamp

Seed:

- default admin
- example team members
- example ignore rules
- example auto-reply rule
- safe default automation settings

Run migrations, seed, read/write tests, then commit.

Phase 3 — Rule and Automation Engine

Model: Sonnet 5

Implement locally:

- normalization
- exact match
- keyword
- multiple keywords
- regex
- sender
- group
- combined conditions
- exception rules
- priority
- team-member filter
- previous/last sender rules
- support escalation
- auto-reply plans
- multi-action rules
- explainable decisions

Actions:

- IGNORE
- TAG
- AUTO_REPLY
- SUPPORT_REQUIRED
- NOTIFY_TEAMS
- NOTIFY_WHATSAPP
- FORWARD
- STOP_PROCESSING

Write comprehensive tests in Bangla, English, Banglish, and mixed messages.

Phase 4 — Functional Dashboard

Model: Sonnet 5

Create real pages:

- Login
- Overview
- WhatsApp Accounts

- Groups
- Internal Team Members
- All Messages
- Needs Attention
- Ignored Messages
- Automation Rules
- Rule Tester
- Automation Control
- Notifications
- Settings
- System Logs

Every button must perform a real action or be omitted.

Phase 5 — OpenWA Provider

Model: Sonnet 5

Implement:
- WhatsAppProvider interface
- OpenWAWhatsAppProvider
- QR login
- QR status to dashboard
- persistent session
- connection lifecycle
- controlled reconnect
- group discovery and sync
- new incoming message events
- outgoing/incoming/system direction handling

Test:
- connect a real account
- restart worker
- restart Docker
- verify session persists
- discover groups
- receive a new group message

If a genuinely difficult OpenWA architecture problem remains, use Opus 4.8 only for that exact issue.

Phase 6 — Message Processing and Safe Automation

Model: Sonnet 5

Implement:

- monitored group check
- duplicate prevention
- message persistence
- team member filtering
- previous sender rules
- default ignore rules
- exception rules
- support escalation
- auto-reply resolution
- idempotency
- cooldown
- per-client limits
- global rate limits
- database-backed outbound queue
- loop prevention
- automation kill switch

Important:

No unrestricted bulk messaging.

No AI API per message.

Do not process outgoing automation as new client requests.

Phase 7 — Notification Providers

Model: Sonnet 5

Implement:

- NotificationProvider
- MicrosoftTeamsProvider
- optional WhatsApp Support Group notification through the existing provider
- test notifications
- retry limits
- failure tracking
- manual retry

Phase 8 — End-to-End Tests

Model: Sonnet 5

Test:

1. Team member message → no client automation.
2. Default message → ignored.
3. Unknown client message → support notification.
4. Support issue → acknowledgement if configured + escalation.
5. Greeting → configured reply.
6. Repeat greeting → cooldown blocks duplicate reply.
7. Duplicate event → no duplicate processing.
8. Worker restart → no duplicate action.
9. Automation paused → no outbound reply.
10. Rate limit reached → safely blocked.
11. Docker restart → database and OpenWA session persist.

Phase 9 — Production Hardening

Model: Opus 4.8 only if needed

Review:

- OpenWA/browser stability
- session persistence
- reconnect
- database consistency
- queue recovery
- idempotency
- rate limits
- security
- secrets
- logs
- health checks
- restart behavior

Fix only evidence-based critical/high issues.

Phase 10 — Documentation and Polish

Model: Fable 5

Create/update:

- README
- ARCHITECTURE
- OpenWA guide
- Windows installation

- Ubuntu installation
- Docker deployment
- team member management
- automation rule guide
- rule testing guide
- safe auto-reply guide
- Teams setup
- backup/restore
- update guide
- troubleshooting

Do not change stable core logic.

Resource-control rules

- Do not repeatedly paste the full Master Prompt.
- For each phase say: "Continue from the existing project and preserve all working functionality."
- Send only the requirements for the current phase.
- Use Opus only for architecture or genuinely difficult bugs.
- Use Fable for small tasks and documentation.
- Commit every stable phase.

One-day priority order

1. Docker + PostgreSQL + Prisma
2. OpenWA QR login + persistent session
3. Group discovery and monitoring
4. New message listener
5. Team member filtering
6. Duplicate prevention
7. Rule + automation engine
8. Safe auto-reply controls
9. Teams notification
10. Dashboard polish and documentation

Core success pipeline

OpenWA Group

↓

New Message

↓

Direction + Team Filter

↓

Duplicate Check

↓

Local Rule Engine

↓

IGNORE / AUTO_REPLY / SUPPORT_REQUIRED

↓

Safety Controls

↓

Teams and/or WhatsApp Support Group Notification

↓

PostgreSQL Logs